

Cross-rank material exchange — Implementation Plan v4

Implementation Plan v4: ONE general material-read path (Coefficient + GridFunction/CVM) at $np > 1$

v4 supersedes v1/v2/v3. It fixes the v3 review (REVIEW_plan_cross_rank_v3_2026-06-06.md, R-301..R-308) and the earlier rounds. The v3 idea — materialize EVERY material into one projected GridFunction — is ABANDONED: it is technically unsound for a material that JUMPS at the fault face (the whole point of bi-material). v4 keeps the user's goal (ONE general workflow, GridFunction/CVM first-class, no per-problem branching) via a single uniform material-read accessor whose ONLY internal branch is the material REPRESENTATION (analytic Coefficient vs sampled GridFunction vs Constant). The flux pool, the shared-neighbour material, and the per-side fault impedances all call that one accessor — uniformly, local AND across partition seams.

Why v3 (project everything to one GridFunction) is wrong — and what v4 does instead

Evidence (verified in-tree): 1. Fault-face-node pollution (R-301). State/material basis is `BasisType::GaussLobatto` (`wave_operator.inl:36`) $\Rightarrow$ nodes sit ON the fault face ($y=0$). `ProjectCoefficient` of a halfspace evaluates the coeff AT $y=0$, where `sign((x-x0).n)>=0` returns “near/strong” for BOTH sides $\Rightarrow$ a far-side element gets strong material on its fault nodes $\Rightarrow$ polluted per-side material. The current code avoids this with an eps-offset (`bimaterial_wave_operator.inl:1028-1048`). 2. Asymmetric-mesh symmetry loss (R-302/R-304). An order-0 (per-element centroid) projection makes the +/- fault elements read their OWN centroids. On an ASYMMETRIC mesh those centroids are at different depths $\Rightarrow$ for a depth profile $Zp_{plus} \neq Zp_{minus} \Rightarrow$ spurious bi-material $\Rightarrow$ seeds the TPV31 σ_n leak. TPV31 mixed-flux runs on the asymmetric **tpv31_50m.msh**, so this is a real regression, not hypothetical. The current code evaluates BOTH sides at the SAME fault QP (offset only along the normal: `offset_ip(..,in1), in2=-in1`) $\Rightarrow$ exactly symmetric on ANY mesh. 3. No single GridFunction can reproduce (1)+(2): a jump at a face the nodes sit on is not representable; an order-0 average/centroid loses the on-any-mesh symmetry.

v4 design: do NOT project. Read each material in its NATIVE representation through one uniform accessor, extended across partition seams: - Constant: the constant. - Coefficient (TPV6 halfspace, TPV31/TPV102 depth profile, analytic CVM): direct `coeff $\rightarrow$ Eval(T, ip) — local element transform, or the PEER element's face-neighbour transform (GetFaceNbrElementTransformation). Globally evaluable, identical on every rank $\Rightarrow$ NO material MPI. Evaluated at the centroid (bulk) or the eps-offset fault QP (fault) $\Rightarrow$ exact symmetry, no pollution, BYTE-EXACT vs the current code (same eval, extended to the peer). - GridFunction (sampled CVM): read the SOURCE GFs (no projection), face-neighbour-exchanged once (ExchangeFaceNbrData), via GetValue(local) / shape $\times$ FaceNbrData(peer) at the same centroid/eps-offset IP. First-class; the only mode that needs a material MPI exchange.`

The WORKFLOW (flux pool, neighbour, fault loops) is uniform — it never asks “which problem?”. The 3-way switch lives ONLY inside the accessor (the material representation, not the problem).

Face-neighbour APIs (verified present)

```
pfes $\rightarrow$ GetFaceNbrElementTransformation(nbr_idx) (fem/pfespace.hpp:486), GetSharedFaceTransformations(sf) $\rightarrow$ Elem2 (face-nbr, valid after pmesh.ExchangeFaceNbrData()), GetFaceNbrFE/GetFaceNbrElementVDofs, ParGridFunction::FaceNbrData()/GetValue(elem,ip), ftr $\rightarrow$ GetElement1IntPoint()/GetElement2IntPoint(), the eps-offset helper offset_ip (bimaterial_wave_operator.inl:1047).
```

Inventory of non-general parts (all routed through the one accessor)

#	Location	Today	v4
A	ExchangeBiMaterialNeighbours_ inl:140	local-as-neighbour stub	MaterialAtNbr_(nbr_idx, centroid)
B	BuildPerFaceBiMaterialFluxMatrices_ inl:442	shared UPWIND flux_nbr from stub	uses corrected (A)
C	BuildPerFaceCentralFluxMatrices_ inl:679	seam_continuous abort	REMOVE
D	BuildPerFaceCentralFluxMatrices_ inl:704,711	central flux_nbr + INERT guard	corrected (A); guard ACTIVE
E	AssignFaultSidePerMaterialImpedances inl:1064	LOCAL interior only; aborts for GridFunction	local+peer via accessor @ eps-offset; ALL modes
F	BuildGodunovFluxPool_ inl:58 (per_elem_lmr_)	EvalAt (aborts for GridFunction)	MaterialAtLocal_(e, centroid) (ALL modes)
G	seam_continuous_ hpp:168/290 + config	gate for (C)	inert/deprecated

Constraints

- One uniform accessor; no per-problem branching. The flux pool, neighbour, and fault all call MaterialAtLocal_/MaterialAtNbr_.
- Byte-exact for Constant + Coefficient (TPV205/BP5/TPV6/TPV31/TPV102): the accessor's local path IS the current At/EvalAt; the peer path is the same eval at the peer's transform/IP (consistent). Fault stays exactly symmetric (eps-offset, both sides at the same fault QP) on ANY mesh. Bulk neighbour for a depth profile changes from local-centroid (stub) to peer-centroid (correct) — a small re-baseline at seams ONLY in the BULK (the fault is unchanged).
- GridFunction (CVM) first-class: NEW capability (currently aborts); gated.
- **--partition-fault-locality** AVAILABLE, NOT REQUIRED.
- No silent fallback / no silent wrong material: a missing face-nbr entry aborts loudly.
- Every change gated by a serial-or-np≤2 unit test (round-2 directive); the np=8 200 m --dry-run is a NON-gating production smoke.
- Build: main repo, conda activate mfem-dev; TOML tests need main-repo toml11 -I; parallel tests mpirun -np 2.

Key math (unchanged; only the material READ is unified + extended to peers)

Shared face L (local) / R (peer); fault QP with +/- from the precomputed *_elem1_on_plus_: - Flux pool / bulk: $(\lambda, \mu, \rho)_e = \text{MaterialAtLocal}_e(e, \text{centroid}_e)$; $(\lambda, \mu, \rho)_R = \text{MaterialAtNbr}_e(\text{nbr_idx}, \text{centroid}_R)$. - Fault per-side: $(\lambda, \mu, \rho)_{\{+/-\}} = \text{accessor at the eps-offset fault QP on the +/- element (peer side via MaterialAtNbr_)}$. Welded star / R,T unchanged (B0).

Phase 1: The uniform material accessor (**MaterialAtLocal_ / MaterialAtNbr_**)

Goal

Two accessors return (λ, μ, ρ) at a given element+IP, locally and across a seam, for all three representations; the GridFunction source GFs are face-nbr-exchanged. Not yet consumed (operator still uses its current reads).

Files to Modify

- dynamic/bimaterial_wave_operator.hpp — declare the accessors + a one-time face-nbr setup flag.

- `dynamic/bimaterial_wave_operator.inl` — implement; call the geometry/GF exchange in the ctor.
- `dynamic/heterogeneous_material.hpp` — (optional) expose the 3 coeff ptrs / GFs already public.

Detailed Requirements

1. Ctor (parallel only), once, on ALL ranks (collective-safe): `pmesh.ExchangeFaceNbrData();` // geometry → peer transforms if `material_→mode == GridFunction: lambda_gf→ExchangeFaceNbrData(); mu_gf→...`; `rho_gf→...`; (or a combined vdim-3 GF — implementer's choice; assert all share one space).
2. `void MaterialAtLocal_(int elem, const IntegrationPoint &ip, real_t &lam, real_t &mu, real_t &rho) const:`
 - Constant: the constant triple.
 - Coefficient: `ElementTransformation *T = mesh_.GetElementTransformation(elem); T->SetIntPoint(&ip); lam = lambda_coef->Eval(*T, ip); ...` (this IS `MaterialField::EvalAt`; may call it directly).
 - GridFunction: `lam = lambda_gf->GetValue(elem, ip); mu = mu_gf->GetValue(elem, ip); rho = rho_gf->GetValue(elem, ip);` (no abort — reads the source GF).
 - `assert rho>0, lam+2mu>0.`
3. `void MaterialAtNbr_(FaceElementTransformations *ftr, const IntegrationPoint &ip_peer, real_t&, real_t&, real_t&) const (peer = Elem2):`
 - Constant: the constant.
 - Coefficient: `ElementTransformation *Tn = ftr->Elem2; Tn->SetIntPoint(&ip_peer); lam = lambda_coef->Eval(*Tn, ip_peer); ...` (`ftr->Elem2` is the face-nbr transform).
 - GridFunction: `nbr_idx = ftr->Elem2No - ne_; fe = (gf space)->GetFaceNbrFE(nbr_idx); Vector shape(fe->GetDof()); fe->CalcShape(ip_peer, shape); Array<int> vd; (gf space)->GetFaceNbrElementVDofs(nbr_idx, vd); const Vector &fnd = lambda_gf->FaceNbrData(); lam = sum_i shape(i)*fnd(vd[i]); ...` per component (assert sizes; the R-004 layout lesson).
 - `assert rho>0, lam+2mu>0.`
4. NO projection anywhere. NO order-0/order-p material GridFunction is created (v3 dropped).

Edge Cases

- Serial: no peer; `MaterialAtNbr_` unused; `ExchangeFaceNbrData` no-op.
- Rank with no shared faces: still calls the ctor `ExchangeFaceNbrData` (collective, R-209).
- GridFunction source space != state space order: abort with a clear message.

Acceptance Criteria

- ☐ Compiles.
- ☐ Gate (np=2) **accessor_local_peer_all_modes** — 2 hexes 1/rank. For EACH mode (Constant; Coefficient 3 non-proportional comps; GridFunction per-element step), assert `MaterialAtLocal_(local, ip) == local` expected AND `MaterialAtNbr_(ftr, ip_peer) == PEER` expected, per component to 1e-12, at BOTH a centroid ip and a near-face (eps-offset) ip. (Covers R-301-class pollution by using a near-face ip on a Coefficient jump — the peer/local values must be the PURE per-side material, not a mix.)
- ☐ Gate (np=2) **accessor_coeff_no_material_mpi** — Coefficient mode performs no material `ExchangeFaceNbrData` (only geometry); assert via a counter/flag.

Dependencies: none. Required by: 2,3,4.

Phase 2: Route flux pool + shared neighbour through the accessor; remove the abort

Files to Modify

- `dynamic/bimaterial_wave_operator.inl:`
 - `BuildGodunovFluxPool_:` `per_elem_lmr[e] = MaterialAtLocal_(e, Geometries.GetCenter(gtype))` (replaces `material.EvalAt`; works for GridFunction now).

- ExchangeBiMaterialNeighbours_: shared_face_neighbour_material_[sf] = MaterialAtNbr(ftr, center_of_peer_geom); delete the stub + the R-004 WARNING.
- BuildPerFaceCentralFluxMatrices_: delete the MFEM_VERIFY(Constant||seam_continuous_) guard.

Acceptance Criteria

- Gate (np=2) **neighbour_matches_accessor_all_modes**: shared_face_neighbour_material_[sf] == MaterialAtNbr(ftr, peer_centroid), all 3 modes.
- Gate (np=2) **shared_central_matches_serial**: a CONTRAST face built central in SERIAL vs np=2 → identical deposit for fixed (Q_self, Q_nbr) to 1e-12 (R-203).
- Gate (np=2) **flux_pool_gridfunction_constructs**: GridFunction ⇒ BuildGodunovFluxPool_ no longer aborts (R-306/F).
- Gate (np=2) **bulk_constant_halfspace_byteexact**: Constant + halfspace ⇒ per_elem_lmr_ and shared_face_neighbour_material_ bit-identical to the current code's values.
- (smoke, NON-gating) np=8 TPV6 mixed-flux --dry-run constructs (ADER and RK).

Dependencies: Phase 1. Required by: 3,4.

Phase 3: Activate the shared-face contrast guard + IMPL-8 invariant

(unchanged from v3) - n_reclass_shared tally (dedup by lower global element id, or print local-exact + shared per-rank; R-005). Contrast guard at inl:711 now sees real contrast. - [] Gate (np=2) **shared_contrast_reclassify_and_impl8**: strong contrast on a shared corridor face ⇒ NOT in central_flux_face_set_ AND per_face_central_flux_.size() == cen (R-202; add a const accessor). - [] Gate (np=2) **shared_weak_contrast_stays_central**. ### Dependencies: 1,2.

Phase 4: Per-side fault material through the accessor (eps-offset; all modes)

Goal

Shared fault faces get correct per-side impedances at the eps-offset fault QP; depth-profile stays EXACTLY symmetric on any mesh; GridFunction faults supported.

Files to Modify

- dynamic/bimaterial_wave_operator.inl AssignFaultSidePerMaterialImpedances:
 - replace the interior material_→EvalAt(ftr→Elem1No, *ftr→Elem1, ip1, ...) and the Elem2 eval with MaterialAtLocal_(ftr→Elem1No, ip1, ...) and the analogous peer read; remove the GridFunction skip at inl:1034 (the accessor handles GridFunction).
 - add a SHARED-fault loop over fault_shared_faces_:
 - * ftr = pmesh.GetSharedFaceTransformations(sf); KEEP the eps-offset: ip1 = offset_ip(*ftr→Elem1, ftr→GetElement1IntPoint(), in1); ip2 = offset_ip(*ftr→Elem2, ftr→GetElement2IntPoint(), in2); (in2 = -in1 — SAME fault QP, opposite normal ⇒ exact symmetry).
 - * LOCAL side: MaterialAtLocal_(ftr→Elem1No, ip1, ...). PEER side: MaterialAtNbr_(ftr, ip2, ...) (Coefficient: coeff at the peer eps-offset via ftr→Elem2; GridFunction: face-nbr GF interp at ip2).
 - * Index spaces (explicit, R-204): e1_plus = shared_fault_elem1_on_plus_[si] (POSITION); base = shared_fault_dof_offset_.at(sf) (RAW-sf); assign +/- from e1_plus; write into dof_data[base+q] as interior does.

Acceptance Criteria

- ☐ Gate (np=2) **shared_fault_symmetric_depthprofile**: depth-profile (f(z)) fault on a seam, ON AN ASYMMETRIC partition $\Rightarrow$ every shared fault DOF $|Zp_plus - Zp_minus| \leq 1e-12 * Zp_plus$ (R-001/R-201/R-302 — the eps-offset keeps it exact on any mesh; FAILS for any per-element /centroid scheme on an asymmetric mesh).
- ☐ Gate (np=2) **shared_fault_contrast**: halfspace fault on a seam $\Rightarrow Zp_minus(\text{strong}) \neq Zp_plus(\text{weak})$, matching the serial interior-fault values to $1e-12$.
- ☐ Gate (np=2) **shared_fault_gridfunction**: GridFunction with a per-element step across the fault $\Rightarrow$ correct per-side contrast (NEW; legacy aborts).
- ☐ Gate (np=2) **shared_fault_index_mapping**: ≥ 2 shared fault faces, distinct contrasts $\Rightarrow$ each DOF carries ITS face's values (R-204).
- ☐ Gate (np=2) **shared_fault_constant_byteexact**: Constant $\Rightarrow Zp_plus == Zp_minus == \text{const.}$

Dependencies: Phase 1.

Phase 5: Deprecate seam_continuous + make fault-locality optional

- seam_continuous_inert (no consumer after Phase 2); kept for config compat.
 - --partition-fault-locality optional; note in jobs/configs.
 - (No separate --material-legacy-eval fallback is needed — the accessor's Coefficient/Constant path IS the legacy eval; it is byte-exact by construction. R-305 is resolved structurally: there is nothing to A/B because v4 does not change the local Coefficient/Constant eval.)
 - ☐ Gate (config-parse) extend seas_test_tpv_config_parse: seam_continuous absent parses; =true still parses (no abort) (R-208).
 - ☐ (smoke, NON-gating) np=8 TPV6 mixed-flux --dry-run WITH/WITHOUT fault-locality, ADER+RK. ### Dependencies: 1-4.
-

Testing Strategy (every change unit-gated; np=8 dry-run = non-gating smoke)

New tests/parallel/test_bimaterial_seam_material_np2.cpp (mpirun -np 2), harness + explicit 2-rank partition from tests/parallel/test_bimaterial_mixed_flux_shared.cpp. Cases ($\rightarrow$ phase):
- P1: accessor_local_peer_all_modes (centroid + near-face ip; all 3 modes — the headline gate), accessor_coeff_no_material_mpi. - P2: neighbour_matches_accessor_all_modes, shared_central_matches_serial, flux_pool_gridfunction_constructs, bulk_constant_halfspace_byteexact. - P3: shared_contrast_reclassify, shared_weak_contrast_stays_central. - P4: shared_fault_symmetric_depthprofile (asymmetric partition), shared_fault_contrast, shared_fault_gridfunction, shared_fault_index_mapping, shared_fault_constant_byteexact. - P5: config-parse seam_continuous. Plus a serial 2-element central/fault parity case in tests/unit/ (the oracle P2/P4 compare to). Makefile: seas_test_bimaterial_seam_material_np2 + mpirun -np 2 target; link as test_bimaterial_mixed_flux_shared.

Regression (NON-gating)

- Constant + Coefficient (TPV205/BP5/TPV6/TPV31/TPV102 FAULT): byte-exact (the accessor's local eval is unchanged; the fault is eps-offset on both sides). BULK neighbour for a depth profile shifts $O(\text{grad} \cdot h)$ at seams (stub $\rightarrow$ correct) — re-baseline that ONE quantity; the fault does not move.

Risk Assessment

- R1 — Coefficient peer eval $\neq$ peer's own value. Identical coeff + same physical point; P1 per-component gate asserts it.
- R2 — GridFunction face-nbr layout. MaterialAtNbr_indexes via GetFaceNbrElementVDofs (R-004 lesson); P1 gate asserts exact peer values incl. a near-face ip.

- R3 — GridFunction fault interpolation at the eps-offset. The source GF read at the eps-offset is sub-element + symmetric (both sides at the same fault QP); P4 shared_fault_gridfunction + shared_fault_symmetric_depthprofile gate it. (If a CVM GF itself encodes a fault jump, the CVM BUILDER must eps-offset-sample — an upstream concern, noted, not in this plan.)
- R4 — index transposition (si vs sf), Phase 4. Explicit + the ≥ 2 -face gate.
- R5 — collective symmetry. pmesh.ExchangeFaceNbrData() (+ the GridFunction GF exchanges) at the ctor on ALL ranks; no per-rank-conditional exchange.
- R6 — ctor order. Exchange setup BEFORE BuildGodunovFluxPool_/ExchangeBiMaterialNeighbours_; AssignFault is post-ctor (driver), peer transforms already valid.
- R7 — DEVIATION from “materialize everything to one GridFunction” (v3 / user). v4 does NOT project Coefficients to a GF (it reads them directly), because a jump-at-face material cannot be projected without pollution or on-asymmetric-mesh asymmetry (evidence above). The user’s GOAL — one general workflow, GridFunction/CVM first-class, no per-problem branching — IS met: the consumers are uniform; only the leaf accessor knows the representation. DECISION (user sign-off 2026-06-06): ACCEPTED — native read (this design). Implement v4. “